Exercise 3: Text-based Software Analysis (2)

Use Jupyter notebook `regex-strengths-and-limitations.ipynb` on GitHub.

1 Strengths

Flexibility

Consider this C program with a license header in the SPDX format:

```
1 // SPDX-License-Identifier: Apache-2.0
2 int main() { }
```

A developer wrote the following regular expression to check that all source code contains a matching license header:

```
1 r"// SPDX-License-Identifier: .+"
```

Now consider this Python program:

```
1 # SPDX-License-Identifier: Apache-2.0
2 def greet:
3     print("Hello")
```

a. Adjust the above regular expression to work on the Python programming language instead of C. State the new regular expression.

b. The Python program contains a syntax error. State the error and explain whether the regular expression checking for license headers would still work.

Source Positions

Consider the following C program:

```
1 int main(void) {
2     // Review the the input; note who who owns it.
3     // Handle each each result.
4     return 0;
5 }
```

This regular expression checks for repeated words: `r"\b([A-Za-z]+)[ \t]+\1\b"`.

a. Use the documentation at `https://docs.python.org/3/library/re.html` to decode what the regular expression checks for. Explain each symbol in sequence.
(`\1` references the match of the first group `([A-Za-z]+)`)

b. In the notebook, method `check_source` prints "`Violation at X: ...`" once per match. Extend it so that it instead prints both the start and the end of a match, in this format:

```
Violation from 31--38: the the
Violation from 51--58: who who
Violation from 82--91: each each
```

Keep one output line per occurrence, including occurrences on the same source line.

2 Limitations

Each regular expression below is intended to check the stated property. For each, supply one string that produces a false positive (match but the intended property is not violated), and one string that produces a false negative (no match even though the intended property is violated). Provide a short string that triggers the wrong result; it need not be a complete or compilable program. Test your strings in the Jupyter notebook.

a. φ_0 = There is no call to function exit

```
r"\bexit\s*\("
```

b. φ_1 = There is no SPDX license identifier comment

```
r"// SPDX-License-Identifier: .+"
```

c. φ_2 = There is no word accidentally repeated in succession

```
r"\b([A-Za-z]+)[ \t]+\1\b"
```